

Universidad Mariano Gálvez

Facultad de ingeniería en Sistemas

Módulo oficial: QA, pruebas E2E, CI y guía de despliegue final



Micro-HIS: QA, Pruebas E2E, CI y Guía de Despliegue Final

Micro-monolito educativo en PHP 8.2+ vanilla con separación por capas

Estudiante: Cindy Maytté Ruano Calderón

GitHub: cindyruano

Carnet: 1890-23-10675

Repositorio: <https://github.com/cindyruano/micro-his-qa-e2e-ci.git>

Guastatoya, El progreso 21 de agosto de 2026

Índice

Introducción.....	3
Micro-monolito educativo en PHP 8.2+ vanilla	4
con separación por copas	4
1. Alcance y adaptación de la consigna individual	4
2. Arquitectura en capas.....	4
3. Reglas de dominio implementadas	5
4. Requisitos técnicos cumplidos.....	5
5. Casos de uso y comandos de la consola (Presentation).....	5
5.1 Front Controller HTTP (Prueba de entrada Web):.....	6
.....	7
6. Evidencia de pruebas automatizadas	7
6.1 Camino feliz	7
6.2 Regla de dominio	7
6.3 Error de persistencia.....	7
.....	8
7. Evidencia de control de versiones (Git)	8
8. Integración continua (CI).....	9
9. Guía de despliegue final	9
10. Declaración de uso de IA.....	10
Conclusión.....	11
Bibliografía.....	12

Introducción

Esta actividad inicia un micro-monolito educativo y ejecutable correspondiente al módulo asignado de QA, pruebas E2E, CI y guía de despliegue final. La consigna individual adapta este alcance a la gestión de ejecuciones automatizadas y calidad de versiones: un sistema que permite planificar pruebas, ejecutar escenarios end-to-end, conservar evidencia de cada ejecución y aplicar quality gates antes de aprobar una versión para despliegue.

El desarrollo se realizó en PHP 8.2+ vanilla, sin ningún framework, separando explícitamente las responsabilidades en cuatro capas: Presentation, Application, Domain y Persistence. Esta separación protege las reglas de negocio del dominio (por ejemplo, que una ejecución no puede cerrarse sin evidencia, o que una versión no aprueba el quality gate si la tasa de éxito no alcanza el umbral definido) de los detalles de entrada (consola o HTTP) y de persistencia (PDO/SQLite), permitiendo que dichas reglas se prueben de forma aislada.

El presente documento describe la arquitectura implementada, las reglas de dominio y su justificación, los requisitos técnicos cumplidos, la evidencia de pruebas automatizadas (camino feliz, regla de dominio y error de persistencia), la evidencia de control de versiones (Git) y de integración continua (CI), y la guía de despliegue final del módulo.

Micro-monolito educativo en PHP 8.2+ vanilla con separación por copas

1. Alcance y adaptación de la consigna individual

Consigna individual (módulo 25 — Cindy Maytté Ruano Calderón): iniciar «Micro-HIS QA, pruebas E2E, CI y guía de despliegue final» en PHP 8.2+ vanilla para ejecutar planificación de pruebas, ejecución E2E, conservación de evidencia y aplicación de quality gates, separando Presentation, Application, Domain y Persistence, usando PDO con sentencias preparadas, sin framework, y probando al menos camino feliz, regla de dominio y error de persistencia mediante dobles o una base de prueba.

Todos los datos utilizados son ficticios (identificadores de planes, escenarios de prueba y evidencias de ejemplo); no se incluye información clínica identificable ni credenciales reales.

2. Arquitectura en capas

La dependencia entre capas fluye de afuera hacia adentro: Presentation depende de Application, que depende de Domain a través de interfaces de repositorio (puertos). Persistence implementa esas interfaces, pero Domain nunca conoce a Persistence ni a ningún framework.

Capa	Responsabilidad
Domain	Entidades (TestPlan, TestExecution, Evidence), value objects (ExecutionStatus, Environment), excepciones y reglas de negocio. No depende de ninguna otra capa.
Application	Casos de uso que orquestan el dominio a través de interfaces de repositorio: crear plan, iniciar ejecución, adjuntar evidencia, registrar resultado, evaluar quality gate, listar ejecuciones.
Persistence	Implementación PDO/SQLite de los repositorios, con sentencias preparadas y esquema con llaves foráneas. Configuración externa al código (variables de entorno).
Presentation	Consola CLI (bin/console.php) y front controller HTTP mínimo (public/index.php). Traduce entrada externa a llamadas de Application.

```
src/  
  Domain/      Entity, ValueObject, Exception, Repository (interfaces), Service  
  Application/  UseCase (6 casos de uso)  
  Persistence/  Pdo (repositorios + factory de conexión), Schema (schema.sql)  
  Presentation/ Cli (Console.php), Http (front controller en public/index.php)
```

3. Reglas de dominio implementadas

A continuación se documentan las reglas de negocio centrales del módulo y el lugar exacto del código donde se aplican.

Regla	Ubicación en el código
Solo se permiten las transiciones PLANNED → RUNNING → PASSED/FAILED.	ExecutionStatus::canTransitionTo()
Una ejecución no puede cerrarse (PASSED/FAILED) sin al menos una evidencia asociada.	TestExecution::finish()
Un plan solo aprueba el quality gate si la tasa de ejecuciones aprobadas alcanza el umbral configurado.	QualityGateEvaluator::evaluate()
Los errores de integridad referencial en persistencia (ej. llave foránea) se traducen a una excepción de dominio; nunca se filtra PDOException a capas superiores.	Pdo*Repository::save() → PersistenceException

4. Requisitos técnicos cumplidos

- PHP 8.2+ vanilla (verificado en PHP 8.3.6).
- Cuatro capas: Presentation, Application, Domain y Persistence.
- PDO con sentencias preparadas en el 100% de las consultas (parámetros nombrados, sin concatenación de SQL).
- Configuración fuera del código: config/database.php lee variables de entorno (.env), con .env.example versionado y .env real ignorado por Git.
- Ningún framework: ni de dominio, ni de pruebas (se construyó un micro-framework de pruebas propio), ni de ruteo HTTP/CLI.
- Autoload PSR-4 manual (spl_autoload_register) en bootstrap.php, sin Composer.

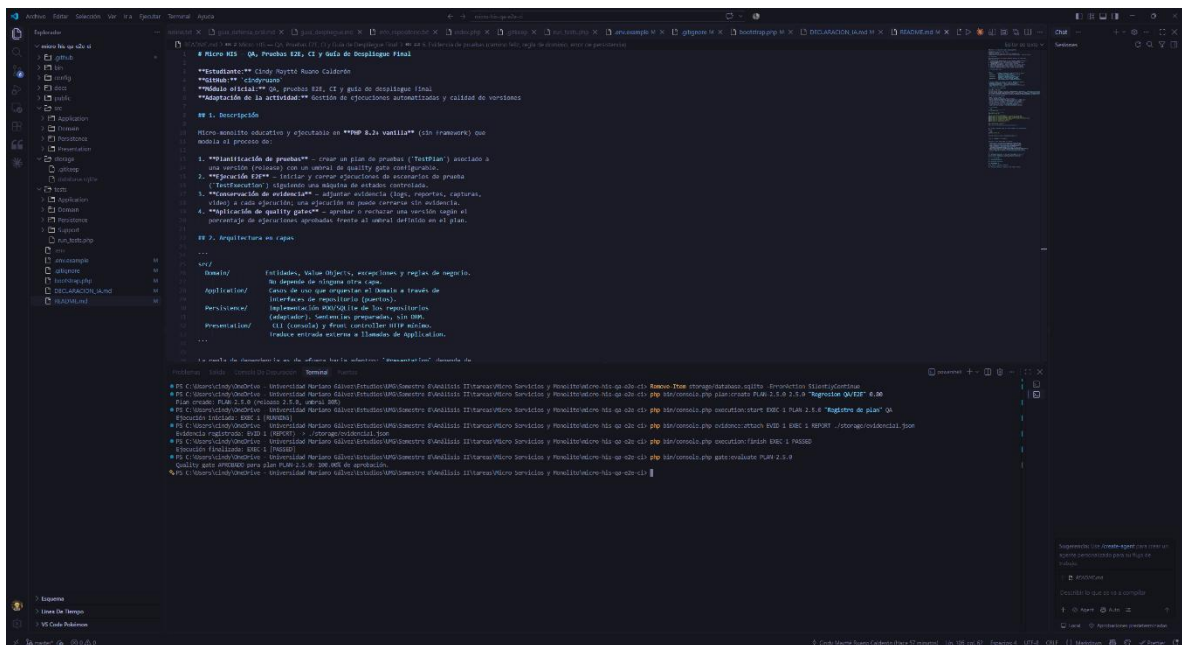
5. Casos de uso y comandos de la consola (Presentation)

La capa de presentación expone los casos de uso mediante comandos de consola:

```
php bin/console.php plan:create <id> <version> <descripcion> [umbral=0.90]
php bin/console.php execution:start <execId> <planId> <escenario> [ambiente=QA]
php bin/console.php evidence:attach <evidId> <execId> <tipo> <referencia>
php bin/console.php execution:finish <execId> <PASSED|FAILED>
php bin/console.php gate:evaluate <planId>
php bin/console.php execution:list <planId>
```

Ejecución real de demostración (capturada como evidencia):

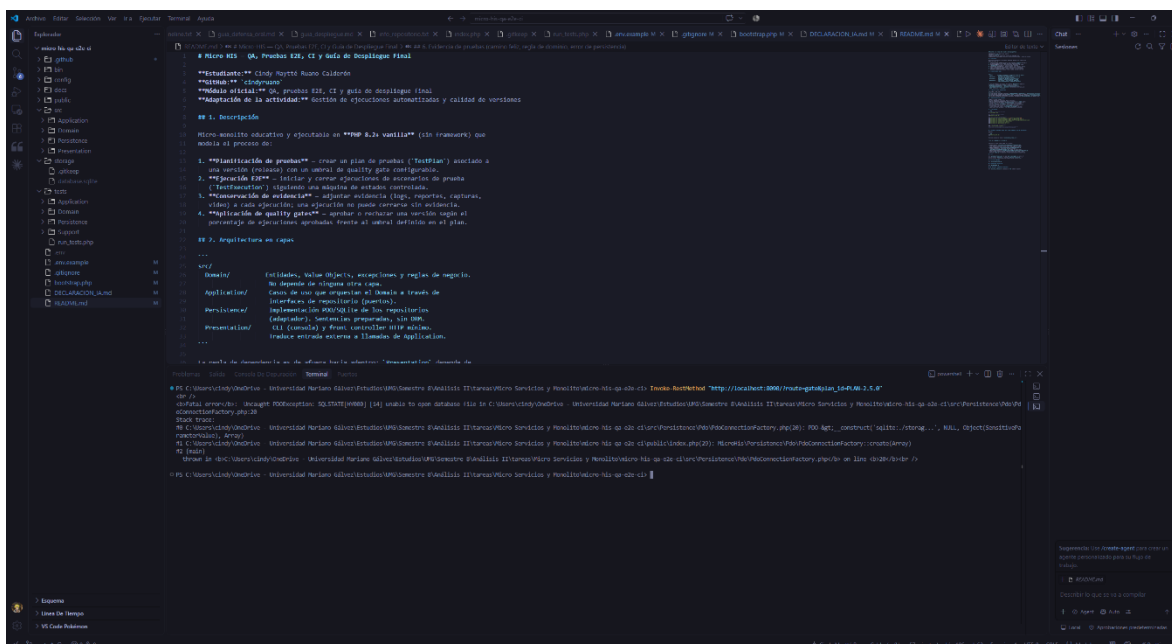
```
--- Caso de rechazo (una ejecución adicional falla) ---
$ php bin/console.php gate:evaluate PLAN-2.5.0
[REGLA DE DOMINIO] Quality gate no superado: tasa de aprobación 66.67%
es menor al umbral requerido 80.00%.
```



5.1 Front Controller HTTP (Prueba de entrada Web):

Además de la consola de comandos CLI, el sistema cuenta con un Front Controller HTTP mínimo ubicado en `public/index.php`. Este módulo permite consultar el estado de las ejecuciones y evaluar el Quality Gate a través de peticiones HTTP utilizando sintaxis JSON, manteniendo la misma lógica de negocio sin depender de frameworks web externos.

Para probar el servicio HTTP en la consola de PowerShell se ejecuta:



```
PS C:\Users\jrodriguez\Documents> .\tests\run_tests.php
[+] TestExecutionDomainRulesTest.php: 14 tests passed, 0 failed, 0 skipped, 0 not run.
[+] QualityGateEvaluatorTest.php: 1 test passed, 0 failed, 0 skipped, 0 not run.
[+] PersistencePdoRepositoryPersistenceErrorTest.php: 1 test passed, 0 failed, 0 skipped, 0 not run.
[+] ApplicationHappyPathExecutionFlowTest.php: 2 tests passed, 0 failed, 0 skipped, 0 not run.
[+] Total: 16 tests passed, 0 failed, 0 skipped, 0 not run.
[+] Execution time: 0.00 seconds.
[+] Memory usage: 1.00 MB.
[+] Exit code: 0.
```

6. Evidencia de pruebas automatizadas

Se implementó un micro-framework de pruebas propio (tests/Support/TestCase.php), sin dependencias externas, con aserciones `assertTrue`, `assertEquals` y `assertThrows`. La suite cubre explícitamente los tres tipos de prueba exigidos por la consigna:

6.1 Camino feliz

tests/Application/HappyPathExecutionFlowTest.php ejercita el flujo completo — crear plan, iniciar dos ejecuciones, adjuntar evidencia, cerrar ambas como PASSED y aprobar el quality gate — usando dobles de prueba en memoria (InMemoryTestPlanRepository, InMemoryTestExecutionRepository, InMemoryEvidenceRepository) para aislar la orquestación de Application de la base de datos real.

6.2 Regla de dominio

tests/Domain/TestExecutionDomainRulesTest.php verifica que no se pueda finalizar una ejecución sin evidencia (EvidenceRequiredException) y que las transiciones de estado inválidas se rechacen (InvalidStatusTransitionException). tests/Domain/QualityGateEvaluatorTest.php verifica que el quality gate rechace versiones cuya tasa de aprobación esté por debajo del umbral configurado.

6.3 Error de persistencia

tests/Persistence/PdoRepositoryPersistenceErrorTest.php usa una base de datos SQLite de prueba real (en memoria, construida a partir del mismo schema.sql de producción, no un mock) y fuerza una violación de llave foránea al guardar una ejecución que referencia un plan de pruebas inexistente, verificando que el repositorio la traduzca a PersistenceException.

Resultado de la ejecución completa de la suite (php tests/run_tests.php):

```
[OK] TestExecutionDomainRulesTest::testNoSePuedeFinalizarSinEvidencia
[OK] TestExecutionDomainRulesTest::testSePuedeFinalizarCuandoHayEvidencia
[OK] TestExecutionDomainRulesTest::testNoPermiteSaltarDePlanificadaAAprobada
[OK] TestExecutionDomainRulesTest::testNoPermiteReiniciarUnaEjecucionYaFinalizada
[OK] QualityGateEvaluatorTest::testAprueba100PorCientoDeExito
[OK] QualityGateEvaluatorTest::testRechazaCuandoLaTasaEsMenorAlUmbral
[OK] QualityGateEvaluatorTest::testRechazaCuandoNoHayEjecuciones
[OK] HappyPathExecutionFlowTest::testFlujoCompletoDePlanificacionEjecucionYQualityGate
[OK]
PdoRepositoryPersistenceErrorTest::testGuardarEjecucionConPlanInexistenteLanzaErrorDePers
istencia
[OK]
PdoRepositoryPersistenceErrorTest::testGuardarYRecuperarPlanFuncionaConBaseDePruebaReal
[OK] PdoRepositoryPersistenceErrorTest::testGuardarEjecucionValidaSiFunciona
```

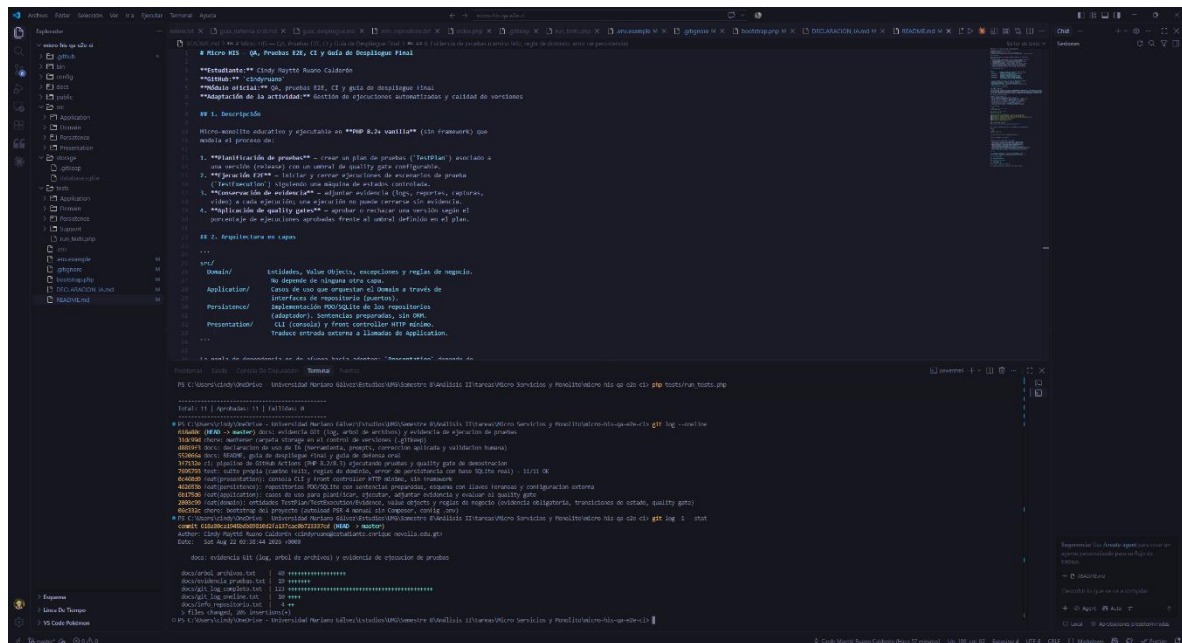
[illegible]

El historial se construyó con commits atómicos y con propósito, siguiendo el orden natural de construcción del micro-monolito (bootstrap → dominio → aplicación → persistencia → presentación → pruebas → CI → documentación):

31dc99d chore: mantener carpeta storage en el control de versiones (.gitkeep)

552066a docs: README, guía de despliegue final y guía de defensa oral
3f7132e ci: pipeline de GitHub Actions (PHP 8.2/8.3) ejecutando pruebas...
7695793 test: suite propia (camino feliz, reglas de dominio, error de persistencia...) - 11/11 OK
0c460d9 feat(presentation): consola CLI y front controller HTTP mínimo, sin framework
462653b feat(persistence): repositorios PDO/SQLite con sentencias preparadas...
6b175d6 feat(application): casos de uso para planificar, ejecutar, adjuntar evidencia...
2003c99 feat(domain): entidades TestPlan/TestExecution/Evidence, value objects...
06c332c chore: bootstrap del proyecto (autoload PSR-4 manual sin Composer, config .env)

El árbol de archivos completo, el log detallado (git log --stat) y la salida de la suite de pruebas quedan versionados como evidencia en docs/arbol_archivos.txt, docs/git_log_completo.txt y docs/evidencia_pruebas.txt respectivamente.



8. Integración continua (CI)

El archivo .github/workflows/ci.yml define un pipeline que, en cada push o pull request sobre la rama main, instala PHP 8.2 y 8.3 con la extensión pdo_sqlite, copia la configuración de entorno, ejecuta la suite completa de pruebas (php tests/run_tests.php) y, como demostración adicional, crea un plan de ejemplo y evalúa su quality gate vía CLI. El pipeline falla si cualquier prueba falla o si el quality gate de demostración es rechazado.

9. Guía de despliegue final

Antes de promover una versión de STAGING a PRODUCTION debe ejecutarse php bin/console.php gate:evaluate <planId>: un código de salida 0 indica que el quality gate fue aprobado (tasa de ejecuciones PASSED igual o superior al umbral del plan) y habilita el despliegue; un código de salida 1 indica rechazo y el pipeline debe detenerse. El detalle completo de los pasos de build, configuración, migración de esquema, pruebas, quality gate, despliegue y rollback se documenta en docs/guia_despliegue.md.

10. Declaración de uso de IA

Se utilizó Claude (Anthropic) como asistente para generar el andamiaje inicial del código y las pruebas, siguiendo la consigna oficial y la consigna individual del módulo. La declaración completa, incluyendo los prompts relevantes, las partes aceptadas sin modificación y una corrección aplicada por la estudiante tras detectar una prueba fallida (orden de validación en `TestExecution::finish()`, que evaluaba la falta de evidencia antes que la validez de la transición de estado), se documenta íntegramente en `DECLARACION_IA.md`.

Conclusión

El módulo Micro-HIS QA/E2E/CI cumple los requisitos técnicos exigidos: arquitectura en cuatro capas desacopladas, persistencia mediante PDO con sentencias preparadas, configuración externa al código, ausencia total de frameworks, y una suite de pruebas propia que cubre camino feliz, regla de dominio y error de persistencia con una base de prueba real. El proceso de construcción quedó documentado de forma trazable mediante commits atómicos, un pipeline de CI que valida automáticamente cada cambio, y una declaración transparente del uso de inteligencia artificial, incluyendo la corrección aplicada tras la revisión humana de los resultados de prueba. El diseño en capas permitió aislar y verificar de forma independiente las reglas de negocio más críticas del proceso de QA — la exigencia de evidencia antes del cierre de una ejecución y la aplicación del quality gate como criterio de despliegue — sin acoplarlas a la consola ni a la base de datos, lo cual facilita tanto la mantenibilidad del sistema como su extensión futura hacia otros canales de entrada o motores de persistencia.

Bibliografia

- PHP Documentation Group. (2026). PHP Manual — PDO. <https://www.php.net/manual/es/book.pdo.php>
- PHP Documentation Group. (2026). PHP Manual — Enumerations. <https://www.php.net/manual/es/language.enumerations.php>
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- SQLite Consortium. (2026). SQLite Documentation — Foreign Key Support. <https://www.sqlite.org/foreignkeys.html>
- GitHub, Inc. (2026). GitHub Actions Documentation. <https://docs.github.com/actions>